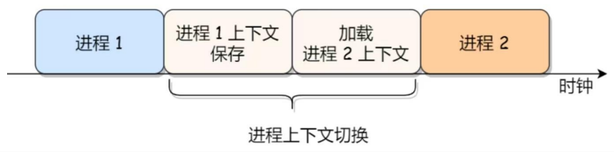
进程与线程

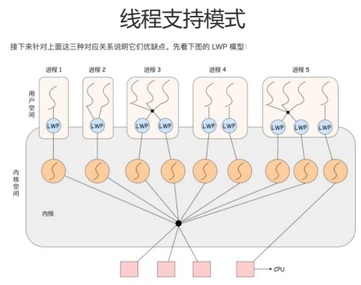
进程

- 进程的定义
  - 对于我们运行一个储存在硬盘中的程序，这个运行中的程序就叫做进程
- 进程的常见状态
  - 运行状态：时刻占用cpu
  - 就绪状态：可运行，由于其他进程出于运行状态而暂时停止运行
  - 阻塞站台：等待某一事件的发生，就开始运行
- 进程的控制
- 进程的上下文切换
  - CPU 的上下文切换
    - CPU寄存器和程序计数是CPU在运行任何任务之前，所必须依赖的环境，这个就叫做CPU 的上下文
  - 进程的上下文切换
    - 进程是由内核管理和调度的，所以进程的切换只能发生在内核态
    - 通常，会把交换的信息保存在进程的PCB中，当要运行另外一个进程的时候，我们需要从这个进程的PCB中取出上下文然后恢复到cpu中，使得这个进程可以继续执行

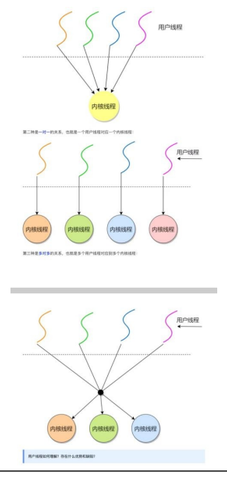


线程

- 线程的定义
  - 线程是进程当中一条执行流程，线程之间可以并发运行且共享相同的地址空间
  - linux中进程和线程都是用tark-struct结构体来表示的
  - 在linux系统中，根据任务的优先级和响应要求，按需要调度
- CPU 如何选择线程
  - 完全公平调度：一般情况下，在优先级相同的情况下，那么都是选择时间最少的依次调度，如果有权重值则需要加权调度
- 线程的选择
  - 线程的优点
    - 一个进程中可以存在多个线程
    - 各个线程之间是可以并发运行的
    - 各个线程之间可以共享地址空间和文件等资源
  - 线程的缺点
    - 当进程中一个线程崩溃的时候，会导致其所属进程的所有线程崩溃
  - 线程和进程的比较
    - 进程是资源分配的单位，线程是cpu调度的单位
    - 进程拥有一个完整的资源平台，线程只是独享必不可少的状态
    - 线程同样也是有就绪，阻塞，执行三种基本状态，同样具有状态之间的转换关系
    - 线程能减少并发执行的时间和空间开销
      - 线程创建时间比进程快
      - 线程终止时间比进程快
      - 同一个进程现场切换比进程快
      - 同一个进程各线程之间共享内存和文件资源
- 线程的分类
  - 用户线程：在用户空间实现的进程，不是由内核管理的线程
    - 优点
      - 他有私有的线程控制块TCB，可以用于不支持线程技术的操作系统
      - 速度比较快。用户线程的切换也是由线程库函数来完成的。无需用户态和内核态的切换。速度快
    - 缺点
      - 因为操作系统不参与导读。如果一个线程发起系统调用堵塞，所有的用户线程都不能执行了
      - 一个线程正在运行，除非他能主动交出cpu使用权。否则他所在的进程无法运行。
      - 由于时间分配给进程，与其他进程相比在多线程执行的时候每个线程得到的时间较慢
  - 内核线程：在内核中实现的线程，是由内核（操作系统）管理的线程
    - 优点
      - 在一个进程中，如果某个内核线程发生系统调用堵塞，并不影响其他内核线程的运行
      - 分配给线程，多线程的进程获得更多的cpu运行时间
    - 缺点
      - 由内核来维护进程和线程的上下文信息。如PCB和TCB
      - 线程的创建。终止和切换都是通过系统调用的方式来进行的，对于系统来说系统开销比较大
  - 轻量级线程：在内核中来支持用户线程
    - 线程支持模式



    - 一对一
      - 优点：并行，当一个线程被堵塞不会影响其他的线程
      - 缺点：创建线程的开销大，每一个用户线程就会产生一个内核线程
    - 多对一
      - 优点：多开没问题，创建效率高
      - 缺点：一个线程j则全都j
    - 多对多
      - 优点：充分结合上面两个优点。完美合理的调度
  - 用户线程和内核线程的对应关系



    - 多对一（用户线程）
    - 一对一（内核线程）
    - 多对多（轻量级线程）

调度

- 触发时机：
  - 状态改变到时候就会触发
    - 就绪到运行
    - 运行到阻塞
    - 运行到结束
- 调度分类
  - 非抢占式算法
    - 一直运行到结束才会调用另外一个
  - 抢占式算法
    - 一个还没运行完就回去调度另外一个
- 原则
  - CPU利用率：保证cpu始终繁忙，提高cpu利用率
  - 系统吞吐量：保证长任务段任务均匀分配。在单位时间内完成的任务数量最多
  - 周转时间：进程运行和阻塞时间综合，一个进程的周转时间越小越好
  - 等待时间：处于就绪队列时间越短越好。
  - 响应时间：用户提交依次请求到相应的时间越短越好
- 算法
  - 算法用来决定优先运行那个进程
  - 先来先服务调度算法（deque）
  - 最短作业优先调度算法（优先选择时间最短的进行）
  - 高响应比优先调度算法（通过优先权进行计算，优先权=（等待时间+要求服务时间）/要求服务时间
  - 时间片轮转调度算法（每一个进程都有一个单位时间，时间一过就交给下一个进程进行）时间一般在20ms-50ms
  - 最高优先级调度算法（通过优先级进行调度）
    - 静态优先级（运行的时候已经确定了优先级）
    - 动态优先级（主要看时间）
    - 也有抢占式和非抢占式之分
  - 多级反馈队列调度算法
    - 设置了多个队列，赋予每个队列不同的优先级，每个**队列优先级从高到低**，同时**优先级越高时间片越短**；
    - 新的进程会被放入到第一级队列的末尾，按先来先服务的原则排队等待被调度，如果在第一级队列规定的时间片没运行完成，则将其转入到第二级队列的末尾，以此类推，直至完成；
    - 当较高优先级的队列为空，才调度较低优先级的队列中的进程运行。如果进程运行时，有新进程进入较高优先级的队列，则停止当前运行的进程并将其移入到原队列末尾，接着让较高优先级的进程运行；
    - 可以发现，对于短作业可能可以在第一级队列很快被处理完。对于长作业，如果在第一级队列处理不完，可以移入下次队列等待被执行，虽然等待的时间变长了，但是运行时间也变更长了，所以该算法很好的**兼顾了长短作业，同时有较好的响应时间**。